



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(ДГТУ)

ОТЧЁТ

по лабораторной работе № 1

**«Подготовка рабочей среды и паспорт
воспроизводимого AI-эксперимента»**

по дисциплине

«Искусственный интеллект в креативных технологиях»

Вариант 2

облачное небо (фон афиши) × число октав +1

Выполнил: Яковенко Максим Михайлович

Группа: Т26ИИКТ-МИК11

Дата выполнения: 24–25 сентября 2026 г.

1. Цель, постановка задачи, критерии успеха и гипотеза

1.1. Цель

Подготовить изолированную рабочую среду, зафиксировать условия эксперимента в паспорте и доказать воспроизводимость результата **измерением, а не утверждением**. Генеративная модель в работе не используется: её роль выполняет учебный генератор процедурных текстур, поэтому предметом изучения является сама практика воспроизводимости, а не качество изображений.

1.2. Постановка задачи по варианту 2

Номер варианта раскладывается по правилу $N = (\text{номер контекста} - 1) \cdot 6 + \text{номер фактора}$. Для $N = 2$ это контекст 1 и фактор 2.

Параметр	Базовое значение	Значение при факторе
Размер изображения, пикселей	96	96
Шаг решётки (cell)	32	32
Число октав	4	5
Затухание (persistence)	0,5	0,5
Квантование, бит	8	8
Интерполяция	сглаженная	сглаженная
Базовый seed S / число seed n	101 / 5	101 / 5

Изменяется ровно один параметр — число октав с 4 на 5. Всё остальное зафиксировано.

1.3. Критерии успеха

Критерии заданы методическими указаниями до начала работы:

1. протокол воспроизводимости выполняется: $EQ_1_2 = \text{true}$, $EQ_1_4 = \text{true}$, $EQ_1_3 = \text{false}$, $EQ_2_4 = \text{true}$;
2. паспорт эксперимента заполнен полностью;
3. вывод о влиянии фактора следует из данных и заранее заданного критерия;
4. ограничения названы.

Учебный критерий существенности: $|\Delta| > 2 \cdot s_base$, где s_base — выборочное стандартное отклонение метрики по базовой серии (делитель $n - 1$).

1.4. Гипотеза, записанная до запуска

Порядок фиксации. Гипотеза и порог записаны в журнал эксперимента и зафиксированы отдельным коммитом в системе контроля версий до получения любых результатов. Порядок «сначала гипотеза, затем эксперимент» подтверждается историей репозитория, а не утверждением в отчёте: коммит 07238eb от 24.09.2026 содержит только журнал с гипотезой, результаты появляются в последующих коммитах.

Механизм фактора. Яркость складывается как $v = \sum a^o \cdot f^o / \sum a^o$ при затухании $a = 0,5$. Добавление пятой октавы вводит поле с шагом решётки $d = \max(1, 32 / 2^4) = 2$ пикселя и весом $a^4 = 0,0625$, а знаменатель нормировки растёт с 1,875 до 1,9375. То есть в кадр добавляется мелкая деталь малого веса, а вклад крупных октав слегка уменьшается.

Предсказания по метрикам:

5. **средняя яркость** — изменения не ожидается: каждая октава имеет среднее около 0,5, нормировка сохраняет масштаб;
6. **границная энергия** — ожидается рост, и это единственная метрика, по которой допускается превышение порога: метрика измеряет средний модуль разности соседних по строке пикселей, а новая октава меняется как раз на масштабе 2 пикселя;
7. **контраст** — ожидается слабое снижение ниже порога: вклад крупных октав, создающих основной размах яркости, уменьшается из-за нормировки;
8. **энтропия** — ожидается слабый рост ниже порога.

Связь с контекстом варианта. Текстура служит фоном афиши под крупный заголовок. Рост граничной энергии означает более дробный фон, конкурирующий с текстом за внимание, то есть довод против добавления октавы для этой задачи.

2. Данные, происхождение, лицензии и ограничения

Входных данных нет. Изображение целиком порождается учебным генератором `lab01_generator.py` версии 1.0 из псевдослучайной последовательности, определяемой значением `seed`. Внешние наборы данных не загружались.

Происхождение кода. Скрипты `lab01_generator.py` и `lab01_experiment.py` выданы преподавателем и не изменялись. Их целостность зафиксирована контрольными суммами в паспорте:

```
lab01_generator.py  sha256 =  
8f07d340a5a765a8e44f1fec407489cf00114ba54216aa7539c4e7116680e152  
lab01_experiment.py  sha256 =  
69257206dab50ef35de448b45764daad0d4f60dbc98febd8953c6faee28e06b5
```

Лицензии. Лицензионные ограничения к синтетическим данным неприменимы: изображения порождены кодом, а не заимствованы. Персональные, конфиденциальные и лицензионно ограниченные материалы в работе не использовались.

Сторонние пакеты не используются. Работа выполнена на стандартной библиотеке Python, поэтому файл `requirements.txt` создан и намеренно оставлен пустым — это и есть зафиксированное состояние среды, а не упущение.

3. Среда, версии и параметры

3.1. Среда части А (локальная)

Параметр	Значение
Операционная система	macOS-26.6.2-arm64-arm-64bit
Архитектура	arm64
Интерпретатор	CPython 3.12.8
Версия zlib	1.2.12
Версия генератора	1.0

Параметр	Значение
Сторонние пакеты	не используются (стандартная библиотека)
Дата и время запуска	2026-09-24 15:00:23 UTC

Версия zlib записана отдельно, потому что от неё зависит отпечаток файла PNG при одинаковых пикселях.

3.2. Структура проекта

```
LR01/
src/          lab01_generator.py, lab01_experiment.py
configs/      параметры варианта
artifacts/    изображения и результаты запусков
reports/      журнал, паспорт, иллюстрации
requirements.txt (пуст: сторонние пакеты не используются)
```

4. Ход работы

4.1. Проверка среды

```
$ python3 --version
Python 3.12.8

$ python3 -c "import zlib, platform; print(zlib.ZLIB_VERSION, platform.platform())"
1.2.12 macOS-26.6.2-arm64-arm-64bit
```

4.2. Намеренная ошибка: seed не задан

Методические указания предписывают воспроизвести типовую причину невоспроизводимости. Генератор запускался дважды без аргумента `--seed`, затем дважды с `--seed 101`.

```
=== Два запуска БЕЗ --seed ===
ПРЕДУПРЕЖДЕНИЕ: seed не задан, использован 562715920 — результат не воспроизводится
seed=562715920 sha256_pixels=f3af13d1997a5a5e
ПРЕДУПРЕЖДЕНИЕ: seed не задан, использован 660758920 — результат не воспроизводится
seed=660758920 sha256_pixels=fb72f468a601e3ef

=== Два запуска С --seed 101 ===
seed=101 sha256_pixels=93fae5913d797284
seed=101 sha256_pixels=93fae5913d797284
```

Симптом. Два запуска одной и той же команды дают разные изображения и разные контрольные суммы.

Диагностика. Сравниваются отпечатки полных массивов, а не картинки на глаз; читается предупреждение скрипта; в коде находится источник случайности — при отсутствии `--seed` значение берётся из системного времени.

Исправление. Всегда передавать `--seed`, записывать его в паспорт и использовать локальный генератор `random.Random(seed)` вместо глобального состояния. Журнал: `reports/error_demo.log`.

4.3. Основной запуск варианта

```
$ python3 src/lab01_experiment.py --variant 2 --out artifacts/v02
```

Запуск создаёт каталог с изображениями, файлами `results.json`, `passport.json` и `passport.md`.

5. Результаты

5.1. Протокол воспроизводимости

Сравниваются отпечатки **полных массивов пикселей**, а не средние значения и не файлы PNG. Схема серии seed: 101, 101, 1101, 101.

Запуск	seed	SHA-256 массива пикселей	SHA-256 файла PNG
1	101	93fae5913d797284	9779f1fe26d3815c
2	101	93fae5913d797284	9779f1fe26d3815c
3	1101	79aa3ff4187bbfe8	68cab27fecf9d4a0
4	101	93fae5913d797284	9779f1fe26d3815c

```
EQ_1_2 = true
EQ_1_4 = true
EQ_1_3 = false
EQ_2_4 = true
```

Все четыре равенства соответствуют требуемым. Запуски 1, 2 и 4 при одном seed побитово совпали; запуск 3 при seed 1101 отличается, что подтверждает: генератор действительно зависит от seed, а совпадение остальных запусков не является следствием кэширования результата.

5.2. Разброс и влияние фактора

Серия из $n = 5$ значений seed (101...105), критерий $|\Delta| > 2 \cdot s_base$ задан до запуска.

Метрика	среднее (база)	s (база)	среднее (фактор)	Δ	$ \Delta /s$	Существенно
средняя яркость	0,4782	0,0481	0,4789	+0,0007	0,02	не обнаружено
контраст	0,1311	0,0102	0,1271	-0,0040	0,39	не обнаружено
граничная энергия	0,0097	0,0006	0,0105	+0,0007	1,29	не обнаружено
энтропия, бит	6,9969	0,1264	6,9545	-0,0423	0,34	не обнаружено

Вывод по каждой метрике: эффект не обнаружен при $n = 5$. Ни одно отношение $|\Delta|/s$ не превысило порога 2. Ближе всех подошла граничная энергия — 1,29.

5.3. Сопоставление с гипотезой

Направление предсказано верно по всем четырём метрикам: граничная энергия выросла, контраст и энтропия слабо снизились, средняя яркость осталась на месте. **Величина предсказана неверно:** ожидалось, что граничная энергия превысит порог, фактически отношение составило 1,29 при пороге 2.

Итог: гипотеза о существенности **не подтверждена при $n = 5$** ; гипотеза о направлении изменений подтверждена. Порог после просмотра результатов не изменялся.

5.4. Пара изображений, seed 101

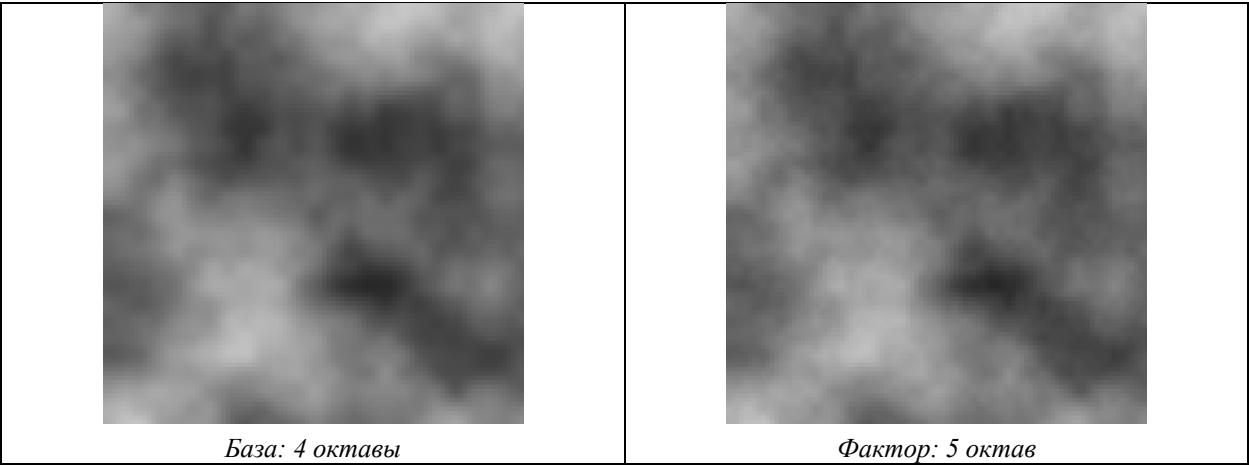


Рисунок 1 — Текстура «облачное небо», seed 101, в одном масштабе. Оба изображения 96 × 96 пикселей, для печати увеличены в 4 раза без интерполяции.

5.5. Проверка диапазона яркости после нормировки

Описание варианта 2 содержит требование отдельной проверки: новая октава берёт следующие числа последовательности, поэтому необходимо убедиться, что **диапазон яркости сохраняется после нормировки**. Проверка выполнена по полным массивам пикселей обеих серий при n = 5.

seed	База: min...max	Размах	Фактор: min...max	Размах	Изменение
101	39...199	160	38...199	161	+1
102	52...209	157	53...207	154	−3
103	43...220	177	43...220	177	0
104	36...216	180	39...210	171	−9
105	64...228	164	66...223	157	−7

Средний размах: база 167,6, фактор 164,0, $\Delta = -3,6$ при $s(\text{база}) = 10,3$, то есть $|\Delta|/s = 0,35$ — заметно ниже порога 2.

Насыщения шкалы нет ни в одном из десяти запусков: минимумы лежат в пределах 36...66, максимумы — 199...228, значения не упираются ни в 0, ни в 255. Нормировка суммой весов Σa° отработала корректно: добавление пятой октавы не сжало и не обрезало диапазон яркости. Риск, указанный в варианте, не реализовался. Машиночитаемый результат: reports/brightness_range_check.json.

5.6. Читаемость заголовка на полученном фоне

Критерий варианта 2 требует обсудить читаемость заголовка по яркостным метрикам. Текстура предназначена под крупный заголовок афиши, поэтому значимы три свойства фона: общий уровень яркости, размах и дробность.

Уровень яркости практически не изменился (0,4782 → 0,4789). Фон остался среднесерым, то есть одинаково неудобным и для чёрного, и для белого текста: заголовок в любом случае потребует подложки, обводки или локального затемнения. Добавление октавы этой проблемы не решает и не усугубляет.

Контраст и размах слегка снизились — контраст с 0,1311 до 0,1271, средний размах со 167,6 до 164,0. Изменения мелкие и ниже порога, но направлены в сторону, полезную для вёрстки: более ровный фон меньше спорит с текстом.

Граничная энергия выросла с 0,0097 до 0,0105 — единственная метрика, отозвавшаяся заметно ($|\Delta|/s = 1,29$ при $n = 5$; по стандартной ошибке разности при $n = 50$ отношение достигает 5,07, см. раздел 5.8). Именно она отвечает за дробность: рост означает больше мелких перепадов яркости на масштабе двух пикселей. Для фона под заголовок это изменение к худшему — мелкая деталь конкурирует с контуром букв, особенно при малом кегле, печати с растром или сжатии изображения.

Вывод по критерию варианта. По яркостным метрикам добавление пятой октавы для данной задачи нейтрально или слегка вредно: уровень и размах яркости сохранились, а единственный устойчивый эффект — рост дробности фона, работающий против читаемости. Оговорка остаётся в силе: четыре метрики описывают статистику яркости и не измеряют читаемость напрямую; строгая проверка потребовала бы макета с конкретным шрифтом и кеглем.

5.7. Воспроизведение демонстрационного варианта в другой среде

Сверх обязательной программы выполнен демонстрационный вариант 1 из методических указаний, чтобы сопоставить свою среду со средой автора демонстрации.

```
$ python3 src/lab01_experiment.py --variant 1 --out artifacts/demo_check
```

Метрика	Моё среднее (база)	Среднее в демонстрации	Совпало
средняя яркость	0,4781768280	0,4781768280	да
контраст	0,1311439849	0,1311439849	да
граничная энергия	0,0097340901	0,0097340901	да
энтропия, бит	6,9968884205	6,9968884205	да

	Среда демонстрации	Моя среда
Операционная система	Windows-10-10.0.19045-SP0	macOS-26.6.2-arm64-arm-64bit
Интерпретатор	CPython 3.13.7	CPython 3.12.8
zlib	1.3.1	1.2.12

Результат: совпадение полное. Все четыре метрики совпали до последнего десятичного знака, отпечатки массивов пикселей и отпечатки файлов PNG совпали тоже.

Интерпретация. Совпадение массивов ожидаемо: генератор использует только метод `random()`, чья последовательность воспроизводится между версиями Python, и арифметику двойной точности. Совпадение отпечатков **PNG** ожидаемым не было: методические указания предупреждают, что байты файла зависят от версии библиотеки сжатия, а версии `zlib` здесь различаются (1.3.1 против 1.2.12). Вывод, который отсюда следует: расхождение

отпечатка PNG **возможно, но не обязательно**, поэтому проверять воспроизводимость нужно по массиву пикселей, а отпечаток файла записывать как дополнительное, менее надёжное свидетельство.

5.8. Проверка при большем числе запусков

Серия повторена при $n = 20$ и $n = 50$, чтобы проверить, изменится ли вывод при увеличении числа запусков.

Метрика	$ \Delta /s$ при $n=5$	$n=20$	$n=50$	$ \Delta /SE$ при $n=5$	$n=20$	$n=50$
средняя яркость	0,02	0,00	0,00	0,02	0,00	0,00
контраст	0,39	0,28	0,24	0,63	0,91	1,20
граничная энергия	1,29	0,94	0,96	2,25	3,15	5,07
энтропия, бит	0,34	0,27	0,23	0,53	0,84	1,13

Учебный критерий не сходится к обнаружению при росте n . Для граничной энергии отношение $|\Delta|/s$ меняется как $1,29 \rightarrow 0,94 \rightarrow 0,96$ и порога не достигает. Причина в устройстве самого критерия: s_base — это разброс **отдельных измерений**, который от числа запусков не уменьшается, а лишь точнее оценивается.

Оценка неопределённости разности средних даёт противоположный результат. Если сравнивать Δ не с разбросом отдельных измерений, а с её собственной стандартной ошибкой $SE = \sqrt{(s^2_{база}/n + s^2_{фактор}/n)}$, отношение для граничной энергии растёт примерно как $\sqrt{n}$: $2,25 \rightarrow 3,15 \rightarrow 5,07$. При этом сама разность устойчива: $+0,00073, +0,00072, +0,00074$ при трёх разных n .

Вывод. Устойчивость Δ при $n = 5, 20$ и 50 — довод в пользу того, что эффект фактора реален, а не является шумом. Учебный критерий не обнаруживает его **по построению**, а не из-за малой серии: он сопоставляет разность средних с разбросом отдельных значений, то есть с величиной заведомо большего порядка. Это и есть предел применимости учебного правила, который следует учитывать при переносе методики на реальные эксперименты.

6. Часть Б. Рабочая область Engее

Часть Б выполнена 25.09.2026 в 21:42 (UTC+3, 18:42 UTC). Учётные данные в отчёт не записываются; адрес страницы среды содержит логин, поэтому снимки экрана сделаны без адресной строки.

6.1. Версии платформы

Что	Наблюдалось	В методических указаниях (19.09.2026)
Личный кабинет	v26.9.1.1	26.9.1
Рабочая среда	Engее 26.9.2-H1	26.8.2-H3

Расхождение зафиксировано, как того требуют указания: платформа обновилась между составлением методички и выполнением работы. Описание интерфейса поэтому сверялось по факту.

6.2. Версия ядра и состав среды

В каталоге /user/ созданы папка с фамилией и подпапка LR01; в ней сохранён скрипт LR01_environment.ngscript со следующей ячейкой:

```
println("VERSION = ", VERSION)
versioninfo()
```

Вывод скопирован дословно (приведён с сокращением списка переменных окружения):

```
VERSION = 1.12.4
Julia Version 1.12.4
Commit 01a2eadb047 (2026-01-06 16:56 UTC)
Platform Info:
  OS: Linux (x86_64-linux-gnu)
  CPU: 112 × Intel(R) Xeon(R) Platinum 8180 CPU @ 2.50GHz
  WORD_SIZE: 64
  LLVM: libLLVM-18.1.7 (ORCJIT, skylake-avx512)
  Threads: 5 default, 3 interactive, 5 GC (on 112 virtual cores)
Environment:
  JULIA_NUM_THREADS = 7,1
  JULIA_DEPOT_PATH = /user/.packages:/opt/depot/core:...
  JULIA_PKG_SERVER = http://pkgserver.pkgserver.svc.cluster.local
  LD_LIBRARY_PATH = /usr/local/nvidia/lib:/usr/local/nvidia/lib64
```

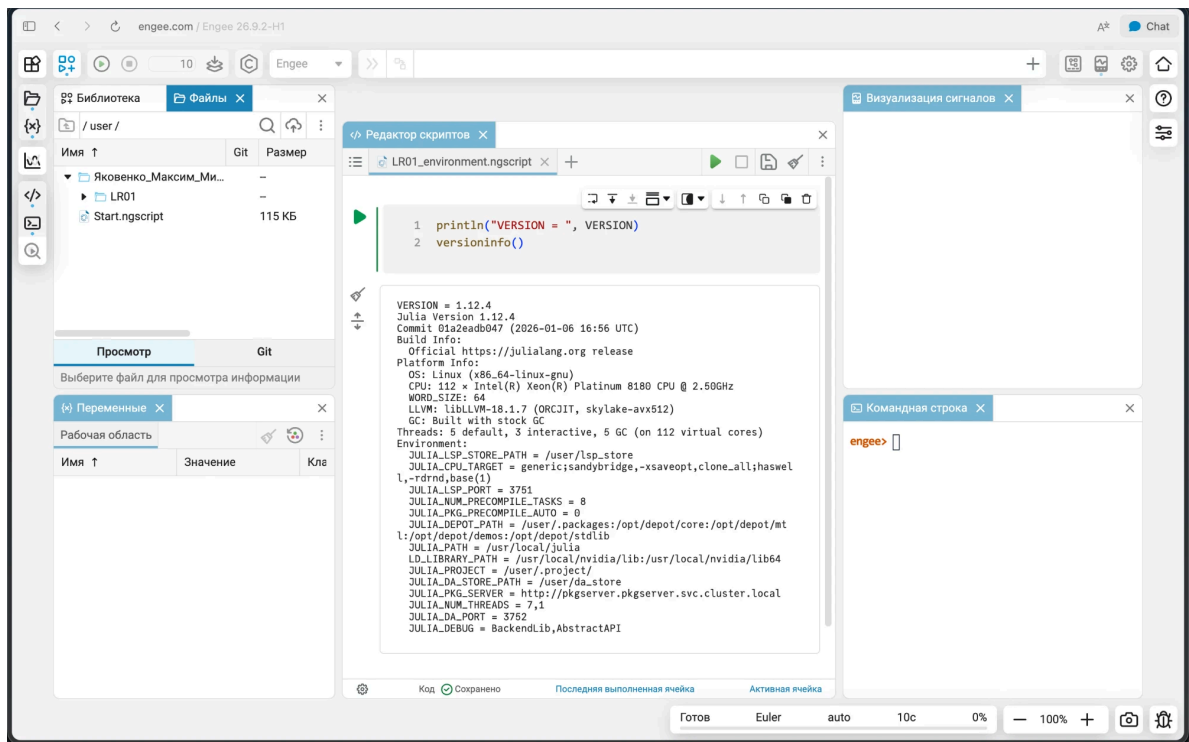


Рисунок 2 — Выполнение ячейки с `VERSION` и `versioninfo()`; панель «Переменные» пуста

Наблюдения, существенные для воспроизводимости:

9. Среда исполняет **Julia 1.12.4 на Linux x86-64**, тогда как часть А выполнена на Python 3.12.8 под macOS arm64. Это другой язык и другая платформа, поэтому паспорт части А к вычислениям в Engage неприменим — потребовался бы отдельный.
10. **Среда многопоточная по умолчанию:** `Threads: 5 default, 3 interactive, 5 GC` при `JULIA_NUM_THREADS = 7,1`. Раздел 2.2 методических указаний прямо предупреждает, что воспроизводимость псевдослучайной последовательности гарантируется, если не работают несколько потоков, разделяющих один генератор. Следовательно перенос эксперимента в Engage потребовал бы либо отдельного генератора на поток, либо явной фиксации числа потоков в паспорте.
11. Машина общая и кластерная (112 виртуальных ядер, `JULIA_PKG_SERVER` указывает на внутренний адрес кластера), поэтому состав ресурсов не контролируется пользователем и может меняться между сессиями.
12. В путях присутствуют библиотеки NVIDIA: среда рассчитана и на вычисления на GPU, хотя в этой работе они не использовались.

6.3. Состояние рабочей области

Отдельной ячейкой выполнено присваивание `seed = 101`, после чего последовательно применены «Очистить все переменные» и «Перезапуск ядра».

Состояние	Таблица «Переменные»	Вывод ячейки в редакторе
После выполнения ячейки	<code>seed = 101</code> , класс <code>Int64</code>	101
После «Очистить все переменные»	пуста	101 — остался на экране
После «Перезапуск ядра»	пуста	101 — остался на экране

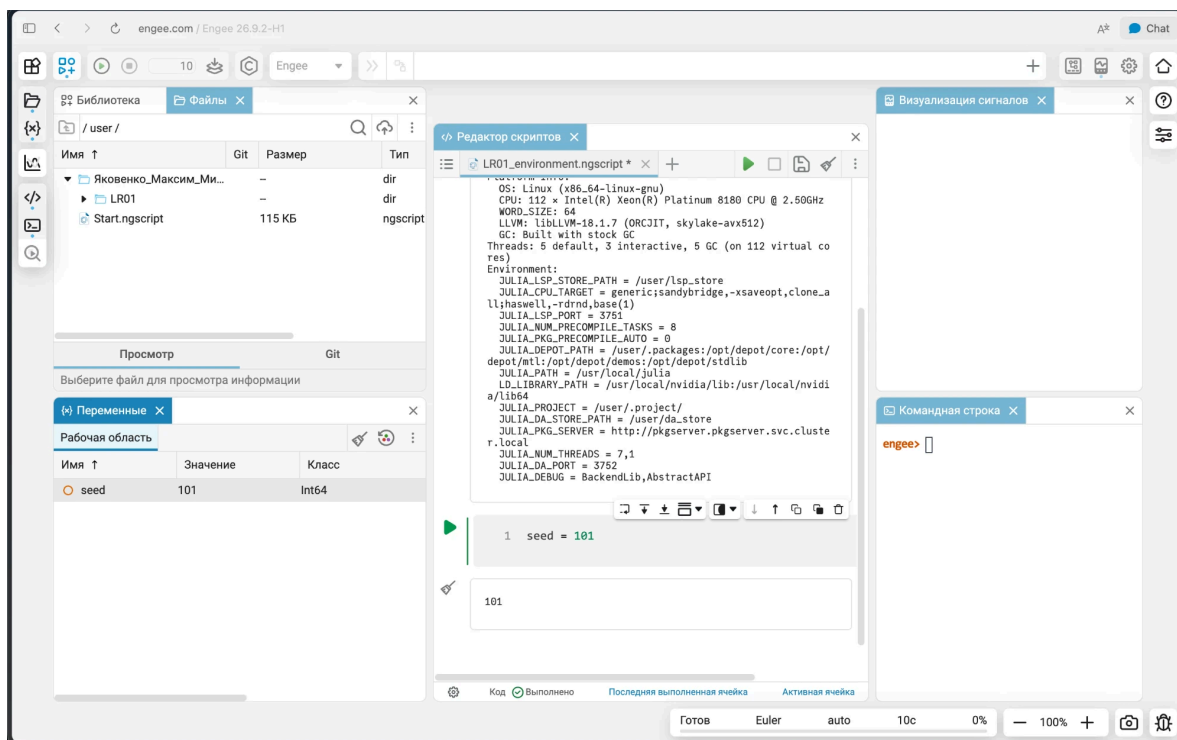


Рисунок 3 — Переменная `seed = 101` в рабочей области

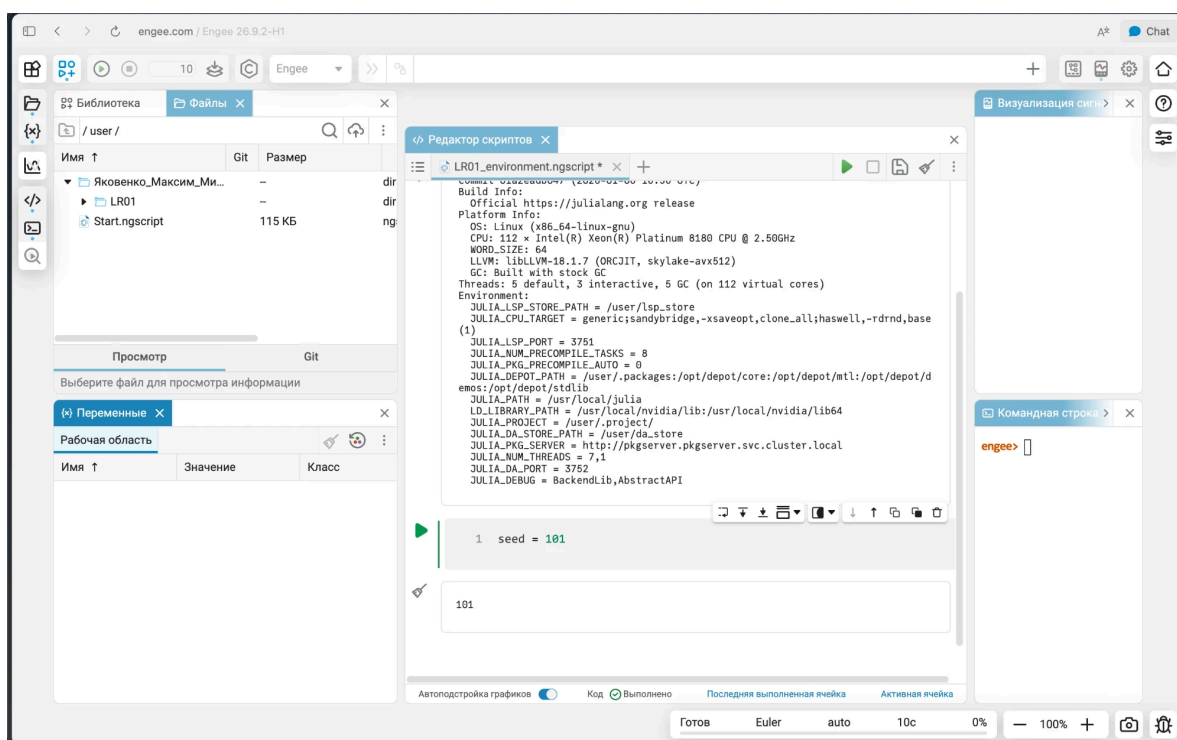


Рисунок 4 — После «Очистить все переменные»: таблица пуста, вывод ячейки сохранился

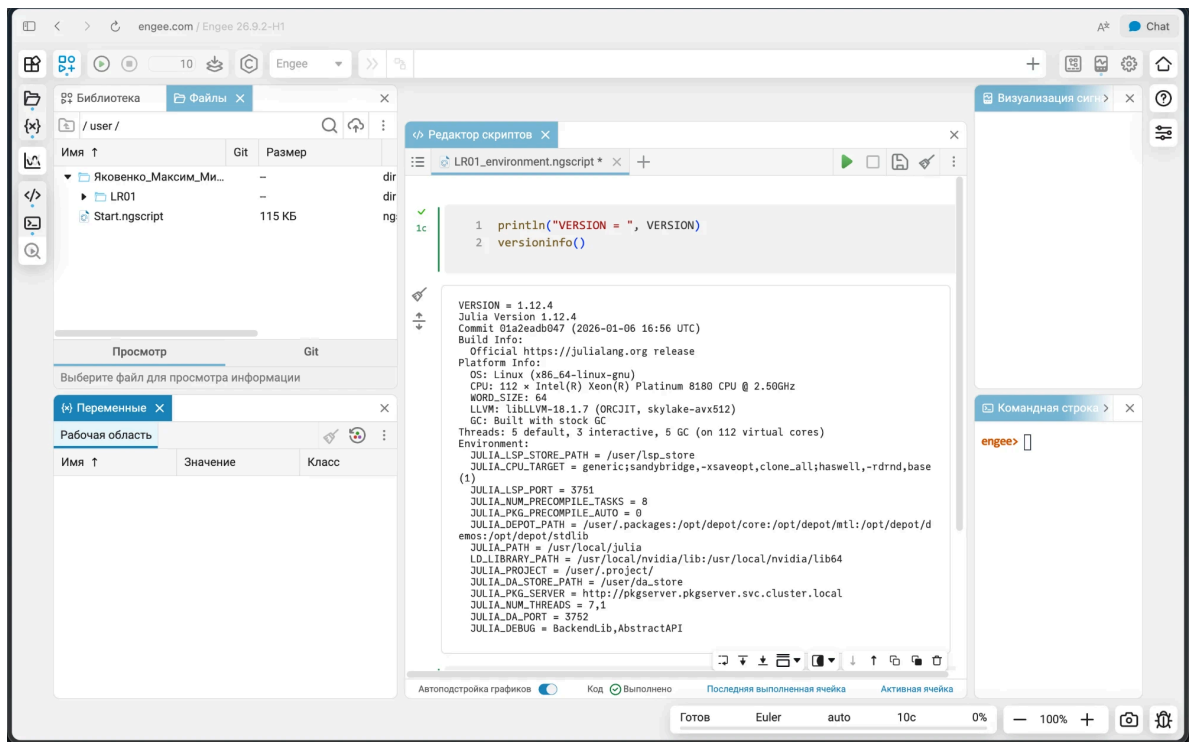


Рисунок 5 — После «Перезапуск ядра»: видимых отличий от предыдущего состояния нет

Три вывода, которые следуют из наблюдения.

13. **Вывод ячейки отражает прошлое, а не настоящее.** Переменной `seed` в рабочей области уже нет, но под ячейкой по-прежнему напечатано 101. Более того, отметка «выполнено за 1 с» у первой ячейки сохранилась после перезапуска ядра, хотя ячейка заново не запускалась. Это штатное поведение всех интерфейсов с ячейками, а не ошибка: вывод и метаданные хранятся вместе с файлом, а состояние ядра живёт отдельно.
14. **Очистка переменных и перезапуск ядра — не одно и то же.** Очистка убирает пользовательские имена из рабочей области, но процесс продолжает работу: остаются загруженные модули, скомпилированный код и глобальное состояние, которого в таблице не видно, — в том числе состояние глобального генератора случайных чисел. Перезапуск уничтожает процесс целиком, о чём среда предупреждает: «Память Engee будет очищена». Отсюда следует практическое правило раздела 2.2 указаний: использовать локальный генератор от явного `seed`, а не глобальное состояние.
15. **Перезапуск ядра визуально неотличим от очистки.** После обеих операций таблица пуста, и среда не даёт признака, что процесс был заменён другим. Значит по внешнему виду рабочей области нельзя судить, с какого состояния начнётся следующая ячейка, и потому в паспорт записывается **порядок выполнения**, а не только содержимое ячеек: один и тот же файл, выполненный в разном порядке, даёт разное состояние.

6.4. Размещение паспорта

В каталог `/user/.../LR01/` загружен файл `passport.md`, рядом сохранён скрипт. Паспорт корректно отобразился средствами среды.

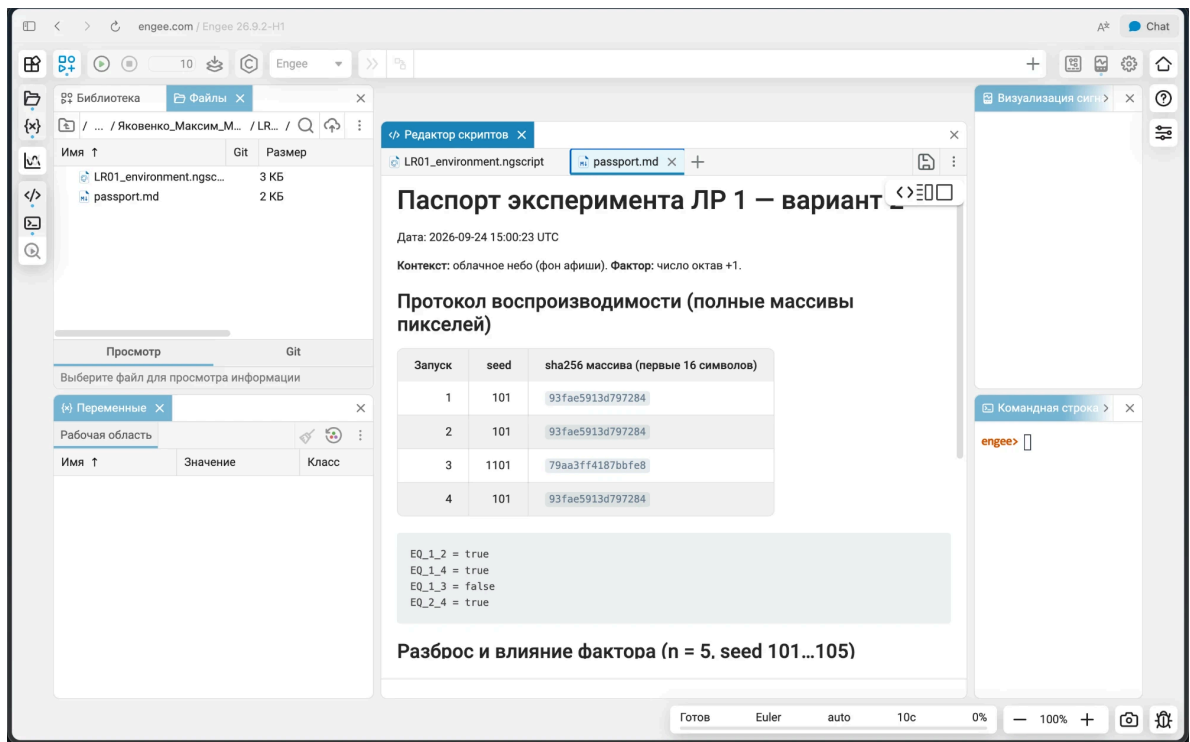


Рисунок 6 — Паспорт эксперимента, загруженный в рабочую область Engage

7. Анализ ошибок, угроз валидности и ограничений

7.1. Ограничения работы

16. **Малая серия.** Основной вывод получен при $n = 5$. При $s_{\text{base}} = 0,0006$ для граничной энергии обнаружить удалось бы лишь эффект, превышающий $2s \approx 0,0012$; фактическая разность $0,0007$ вдвое меньше.
17. **Одна текстура и один базовый seed.** Проверен контекст «облачное небо» при $S = 101$. Перенос вывода на другие контексты не обоснован.
18. **Учебный критерий не является статистическим тестом.** Правило $|\Delta| > 2s$ задано методикой как упрощённое; раздел 5.8 показывает его конкретный предел применимости.
19. **Метрики описывают статистику яркости, а не восприятие.** Вывод о пригодности фона для афиши по ним делать нельзя: четыре числа не измеряют читаемость заголовка.
20. **Отсутствие обнаруженного эффекта не доказывает его отсутствия.** Раздел 5.8 прямо показывает обратное: эффект граничной энергии устойчив по величине, но учебным критерием не выявляется.

7.2. Угрозы валидности и принятые меры

Угроза	Мера
Совпадение отпечатков получено тривиально, без фактического вычисления	Запуск 3 выполнен при другом seed и дал другой отпечаток; равенство $EQ_1_3 = \text{false}$ подтверждает, что результат зависит от seed
Воспроизводимость проверена по среднему значению вместо полных данных	Сравниваются отпечатки SHA-256 полных массивов пикселей; средние значения в протокол не входят
Гипотеза подогнана под результат	Гипотеза и порог зафиксированы отдельным коммитом до запуска; порядок подтверждается историей репозитория
Выборочное стандартное отклонение рассчитано с неверным делителем	Используется делитель $n - 1$; значения берутся из results.json, а не пересчитываются вручную
Результат зависит от неучтённых свойств среды	Паспорт фиксирует ОС, архитектуру, версию интерпретатора, версию zlib и контрольные суммы скриптов; сопоставление с другой средой выполнено в разделе 5.7

7.3. Разобранная ошибка

Типовая причина невоспроизводимости — незаданный seed — воспроизведена намеренно (раздел 4.2). Симптом, диагностика и исправление описаны там же. Эта ошибка характерна и для реальных генеративных моделей: пропущенный seed, глобальное состояние, переиспользованное другой библиотекой, либо параллельные потоки, разделяющие один генератор. Наблюдение части Б подтверждает актуальность последнего пункта: рабочая среда Engae по умолчанию многопоточная.

8. Вывод по цели

Цель достигнута. Подготовлена изолированная среда, её состояние зафиксировано; условия эксперимента записаны в паспорт; воспроизводимость подтверждена измерением, а не утверждением.

Что доказано данными:

21. протокол воспроизводимости выполнен полностью: $EQ_1_2 = true$, $EQ_1_4 = true$, $EQ_1_3 = false$, $EQ_2_4 = true$;
22. при $n = 5$ влияние фактора «число октав +1» не обнаружено ни по одной из четырёх метрик по заранее заданному критерию $|\Delta| > 2 \cdot s_base$;
23. направление изменений предсказано гипотезой верно по всем четырём метрикам, величина — неверно для граничной энергии;
24. результат демонстрационного варианта воспроизведён в другой операционной системе, при другой версии интерпретатора и другой версии zlib — совпадение полное, включая отпечатки файлов PNG;
25. при $n = 20$ и $n = 50$ разность средних для граничной энергии устойчива, а отношение к её стандартной ошибке растёт как $\sqrt{n}$, что указывает на реальность эффекта, не выявляемого учебным критерием.

Чего результат не доказывает: применимости вывода к другим контекстам, факторам и значениям seed; отсутствия эффекта фактора как такового; пригодности текстуры как фона афиши, поскольку читаемость заголовка выбранными метриками не измеряется.

Предложение по доработке методики. Учебный критерий $|\Delta| > 2 \cdot s_base$ сопоставляет разность средних с разбросом отдельных измерений, то есть с величиной заведомо большего порядка, и потому не становится чувствительнее при росте n . Для учебных целей достаточно дополнить его сравнением Δ со стандартной ошибкой разности $SE = \sqrt{(s^2_{база}/n + s^2_{фактор}/n)}$: вычисление не сложнее, а поведение при росте n становится содержательным и демонстрирует связь числа запусков с обнаружимым эффектом.

9. Артефакты, паспорт и ссылки

Репозиторий работы: <https://github.com/ykvnkm/ai-in-creative-industries> — каталог LR01/.

Архив артефактов: LR01_вариант2.zip — содержит все перечисленные ниже файлы без виртуального окружения и служебных каталогов.

Файл	Содержимое
Яковенко_Максим_Михайлович_Вариант_2_ЛР_1.docx	настоящий отчёт
Яковенко_Максим_Михайлович_Вариант_2_ЛР_1.pdf	тот же отчёт в формате PDF
artifacts/v02/passport.json, passport.md	паспорт эксперимента, вариант 2
artifacts/v02/results.json	полные результаты: протокол, метрики, среда
artifacts/v02/artifacts/*.png	изображения четырёх запусков протокола, база и фактор
artifacts/v02_n20/, artifacts/v02_n50/	серии при $n = 20$ и $n = 50$
artifacts/demo_check/	воспроизведение демонстрационного варианта 1

Файл	Содержимое
reports/journal.md	журнал эксперимента с гипотезой, записанной до запуска
reports/error_demo.log	журнал намеренной ошибки «seed не задан»
reports/brightness_range_check.json	проверка диапазона яркости (риск варианта), раздел 5.5
reports/figures/	иллюстрации отчёта
src/lab01_generator.py, src/lab01_experiment.py	скрипты, выданные преподавателем
requirements.txt	пуст: сторонние пакеты не используются

Порядок фиксации гипотезы. Коммит 07238eb от 24.09.2026 содержит журнал с гипотезой и порогом и не содержит результатов; результаты добавлены последующими коммитами. Это позволяет проверить независимо, что порог не изменялся после просмотра данных.

Приложение А. Ответы на контрольные вопросы

Вопросы приведены по разделу 9 методических указаний. Ответы опираются на данные настоящей работы.

1. Почему результат одного запуска нельзя считать результатом эксперимента?

Один запуск — это одна реализация случайного процесса. По ней невозможно отличить влияние изучаемого фактора от обычного разброса между значениями seed: любое различие двух одиночных запусков может оказаться шумом.

Вместо одного числа сообщается: число запусков n , перечень использованных seed, среднее и **выборочное** стандартное отклонение метрики, а также критерий сравнения, заданный до эксперимента. В настоящей работе для граничной энергии это $n = 5$, seed 101...105, среднее 0,0097 при $s = 0,0006$ — и только на фоне этого s разность 0,0007 приобретает смысл.

2. Расчёт среднего и выборочного s ; существенна ли разница

Дано: 0,71; 0,74; 0,69; 0,73; 0,70. Вторая серия со средним 0,77.

```
среднее = (0,71 + 0,74 + 0,69 + 0,73 + 0,70) / 5 = 3,57 / 5 = 0,714
отклонения: -0,004 +0,026 -0,024 +0,016 -0,014
сумма квадратов = 0,001720
s = sqrt(0,001720 / (5 - 1)) = sqrt(0,000430) = 0,0207
2s = 0,0415
Δ = 0,770 - 0,714 = 0,0560
|Δ| = 0,0560 > 2s = 0,0415 -> существенно, |Δ|/s = 2,70
```

Разница существенна по учебному критерию. Необходимые оговорки: серия мала ($n = 5$), поэтому сама оценка s неточна; критерий $|\Delta| > 2s$ — учебное правило, а не статистический тест, он не даёт уровня значимости; сравнивается одна метрика, а при нескольких метриках вероятность случайного превышения порога растёт; корректная формулировка — «эффект обнаружен при $n = 5$ », а не «доказан». Отдельно отмечу: делитель $n - 1$ обязателен, с делителем n получилось бы $s = 0,0185$ и порог 0,0370, то есть вывод остался бы тем же, но оценка разброса была бы смещённой.

3. Почему воспроизводимость проверяют по отпечатку полного массива

Среднее — свёртка, которая теряет почти всю информацию: бесконечно много разных изображений имеют одно и то же среднее. Каноничный пример: изображение из двух пикселей (0; 255) и изображение (127; 128) имеют одинаковое среднее 127,5, но не имеют между собой ничего общего.

Данные настоящей работы показывают то же на реальных изображениях. Запуски 1 (seed 101) и 3 (seed 1101) дают заведомо разные картинки — отпечатки 93fae5913d797284 и 79aa3ff4187bbfe8. Их средние яркости равны 0,4502 и 0,4870, то есть различаются на 0,037 — **меньше, чем разброс между seed** ($s = 0,0481$). Сравнение по среднему признало бы эти изображения неразличимыми; сравнение по отпечатку полного массива различает их немедленно и без порогов.

4. Почему отпечаток PNG может различаться при одинаковых пикселях

PNG хранит сжатые данные. Байты файла зависят от реализации сжатия: версия библиотеки `zlib`, уровень сжатия, выбор фильтров строк. При тех же пикселях другая версия библиотеки может дать другую последовательность байтов и, следовательно, другой отпечаток файла.

В паспорт записываются: версия `zlib` (здесь 1.2.12), отпечаток **массива пикселей** как основное свидетельство воспроизводимости и отпечаток **файла** как дополнительное. Настоящая работа даёт по этому вопросу конкретное наблюдение: при сопоставлении с демонстрационной средой версии `zlib` различались (1.3.1 против 1.2.12), а отпечатки PNG тем не менее совпали. Значит расхождение файла **возможно, но не обязательно**, и строить на нём проверку воспроизводимости нельзя ни в одну, ни в другую сторону.

5. Что означает $s = 0$ для метрики

Формально — все n значений совпали. На практике это почти всегда признак ошибки постановки, а не свойства данных:

- 26.серия фактически состоит из одного запуска, повторённого n раз, — например, `seed` не менялся внутри цикла;
- 27. $n = 1$, и выборочное s не определено (деление на $n - 1 = 0$);
- 28.метрика вырождена для данных параметров: например, при квантовании до 1 бита или константном изображении энтропия и контраст могут совпасть во всех запусках;
- 29.ошибка агрегации: в список попало одно и то же значение.

Проверка. Вывести список фактически использованных `seed` и отпечатки массивов пикселей по каждому запуску. Если отпечатки совпадают — серия вырождена, менялся не `seed`, а что-то другое либо ничего. Если отпечатки различаются, а метрика одинакова — метрика действительно нечувствительна к этим данным, и это содержательный результат, который следует записать.

6. Как найти причину невоспроизводимости, если код не менялся

Разбор на примере намеренной ошибки из раздела 4.2 настоящей работы: две одинаковые команды дали разные изображения при неизменном коде.

- 30.**Сравнить отпечатки полных данных, а не картинки на глаз.** Получено `f3af13d1997a5a5e` и `fb72f468a601e3ef` — различие объективно, спорить не о чем.
- 31.**Прочитать вывод программы.** Скрипт печатал предупреждение: «`seed` не задан, использован 562715920 — результат не воспроизводится». Сообщения об ошибках и предупреждения читаются раньше, чем начинается поиск причины.
- 32.**Найти источник случайности в коде.** При отсутствии `--seed` значение берётся из системного времени, то есть различается между запусками — значит меняется не код, а вход.
- 33.**Проверить гипотезу.** Два запуска с явным `--seed 101` дали один и тот же отпечаток `93fae5913d797284`. Причина подтверждена.

Общий принцип: если код неизменен, различие вносит что-то вне кода — вход, состояние процесса, среда или параллелизм. Перебирать эти источники следует в порядке убывания вероятности, и первым проверяется `seed`.

7. Почему локальный `random.Random(seed)`, а не глобальный `random.seed(seed)`

Глобальное состояние доступно всему процессу. Любая импортированная библиотека может вызвать `random.random()` и сдвинуть последовательность; тогда результат начинает зависеть от того, какие ещё модули загружены и в каком порядке они обращались к генератору. Это скрытая зависимость: в коде эксперимента её не видно.

При двух потоках порядок обращений к общему генератору не определён планировщиком. Потоки разбирают числа из одной последовательности в непредсказуемом порядке, и повторный запуск с тем же `seed` даёт другой результат. Документация Python прямо оговаривает воспроизводимость условием отсутствия нескольких потоков.

Локальный экземпляр `random.Random(seed)` принадлежит только тому коду, который его создал, и не разделяется с другими частями программы. Наблюдение части Б показывает, что риск не теоретический: рабочая среда Engee многопоточная по умолчанию — `Threads: 5 default, 3 interactive, 5 GC` при `JULIA_NUM_THREADS = 7,1`.

8. Поля паспорта и потеря, которая сильнее всего мешает через полгода

Раздел паспорта	Что записывается
Цель и критерий	что исследуется; по какому признаку и порогу принимается решение
Данные	источник, лицензия, контрольная сумма (здесь данных нет: синтетический генератор)
Модель	название, версия, контрольные суммы скриптов
Параметры	все параметры генератора, <code>seed</code> , число запусков
Среда	ОС, версия интерпретатора, версии пакетов, версия <code>zlib</code>
Процедура	точная команда запуска
Результаты	контрольные суммы, метрики со средним и разбросом, вывод
Ограничения	что не проверялось; известные несоответствия

Самая тяжёлая потеря — `seed`: без него результат не восстановим в принципе, никакая остальная информация не помогает. Методические указания относят результат без записи `seed` к критическим ошибкам.

Если же `seed` записан, то через полгода сильнее всего мешает **отсутствие версий среды**. Причина в том, что версии меняются молча: интерпретатор обновился, библиотека подтянулась новее, машина другая — и расхождение результата объяснить нечем, потому что не с чем сравнивать. `Seed` забывают заметно и сразу, версии — незаметно и с отложенным эффектом. Подтверждение из смежной работы курса: при одинаковых `model_id`, `revision`, `prompt` и `seed` два прогона `text-to-image` дали разные контрольные суммы, и различались только устройство и тип данных — то есть ровно те поля, которые легко не записать.

9. Почему `requirements.txt` содержит точные версии, а не диапазоны

Точная версия (`=`, как её выводит `pip freeze`) фиксирует **состояние** среды. Диапазон (`>=`, `~`, `^`) фиксирует не состояние, а **правило сборки**: он описывает, что считать допустимым, а не что было установлено.

Что произойдёт при диапазоне. Через месяц выйдет новая версия, подходящая под диапазон, и установка из того же файла даст другое окружение — без единого изменения в самом файле. Результат может измениться, а причину будет не найти: файл, который должен был зафиксировать среду, покажет одно и то же в обоих случаях. Это ровно тот тип скрытой зависимости, о котором пишет Sculley и др.: связь есть, но она не выражена явно.

В настоящей работе `requirements.txt` создан и пуст, поскольку сторонние пакеты не используются. Пустой файл здесь — тоже точная фиксация: он утверждает, что установленных зависимостей нет, и это проверяемо. Фиксация версии интерпретатора при этом остаётся обязательной и вынесена в паспорт.

10. Какие параметры нужно записать при запуске модели «текст → изображение»

Помимо полей, уже перечисленных в вопросе 8, для генеративной модели добавляются следующие.

Параметр	Почему без него запуск не восстановит
<code>model_id</code> и полная <code>revision</code>	имя модели указывает на репозиторий, а не на состояние весов: владелец может обновить снимок, и тот же код даст другой результат
Точный текст запроса	запрос — это спецификация входа; пунктуация и порядок слов влияют на результат
<code>seed</code> и устройство генератора	генераторы разных устройств дают разные последовательности; генератор на CPU сопоставим между устройствами
Число шагов, <code>guidance scale</code> , размер	определяют режим инференса; допустимые диапазоны зависят от модели
Планировщик (<code>scheduler</code>) и его параметры	траектория расшумления задаётся планировщиком, при его замене результат меняется полностью
<code>device</code> и <code>dtype</code>	<code>float16</code> против <code>float32</code> — другая точность, то есть другой результат, а не тот же быстрее
Вариант весов (<code>fp16</code> / <code>fp32</code>)	в репозитории модели лежат разные файлы весов под один и тот же <code>model_id</code>
Версии библиотек инференса	смена версии конвейера меняет результат при неизменных весах
Число потоков на CPU	задаёт порядок редукций и потому влияет на младшие биты результата
Наличие фильтра безопасности	меняет состав конвейера и влияет на то, будет ли выдан результат вообще

Какие из них забывают чаще всего. По моему опыту выполнения работы с моделью `stabilityai/sd-turbo` — три поля: `revision` (ограничиваются именем модели), `dtype` вместе с вариантом весов (он появляется неявно, как следствие выбора устройства) и число потоков на CPU. Последнее проверено прямым экспериментом: два прогона на одной машине с одинаковыми моделью, ревизией, запросом и `seed` дали разные контрольные суммы — различалось только число потоков PyTorch. После его закрепления отпечатки совпали побитово.

Более общий вывод, к которому приводит эта практика: воспроизводимость — свойство не модели и не `seed`, а **зафиксированной среды целиком**. `Seed` и ревизия задают точку старта; повторить результат байт в байт позволяет только полный паспорт.